

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 19/30

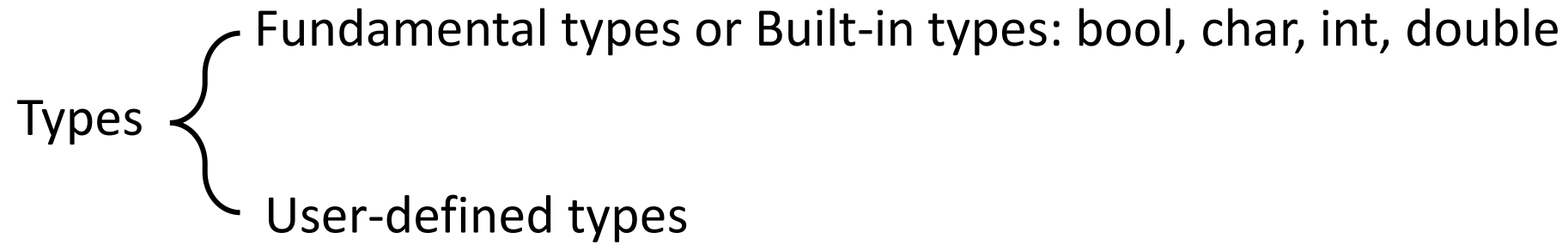
Session 19. Introduction to Classes

- Fundamental types vs. User-defined types
- The C++ classes classification: Concrete and Handle classes
- Classes: Data members and Member functions
- References and further readings
- Q&A

120 min (incl. Q & A)



Types and user-defined types



- A type is a concrete representation of a concept.

Ex. C++ built-in type float $\approx$ real numbers

- The designer of a general-purpose programming language cannot foresee the detailed needs of every application.

C++ class $\longrightarrow$ Creating new types

“Those types are not “abstract”; they are as real as `int` and `float`.”

- Doug McIlroy

• Ideally, such types should not differ from built-in types in the way they are *used*, only in the way they are *created*.

User-defined types: benefits

- C++ is a type-full programming language
- Defining a new type:
 - Separate implementation details from interface
 - Better expressiveness: Problem domain vs. Solution domain
 - Maintainability: Easier to understand and easier to modify programs

Sentence, String, Paragraph, ... → Text processing

Point, Line, Rectangle, Circle, Shape, ... → Graphics

Button, Label, Textbox, Combobox, Icon, ... → GUI

Explosion, Orc warrior, ... → Video games

Cclasses: definitions

A Class defines an object's interface and implementation. It specifies the object's internal representation and defines the operations the object can perform.

GOF: Design Patterns,
Elements of Reusable Object-
Oriented Software

A class is a set of objects that share a common structure and a common behavior.

Grady Booch: Object-Oriented Analysis &
Design with Applications

Class

C++ standard working draft:

A class is a type.

Simula

C++

Bjarne Stroustrup:

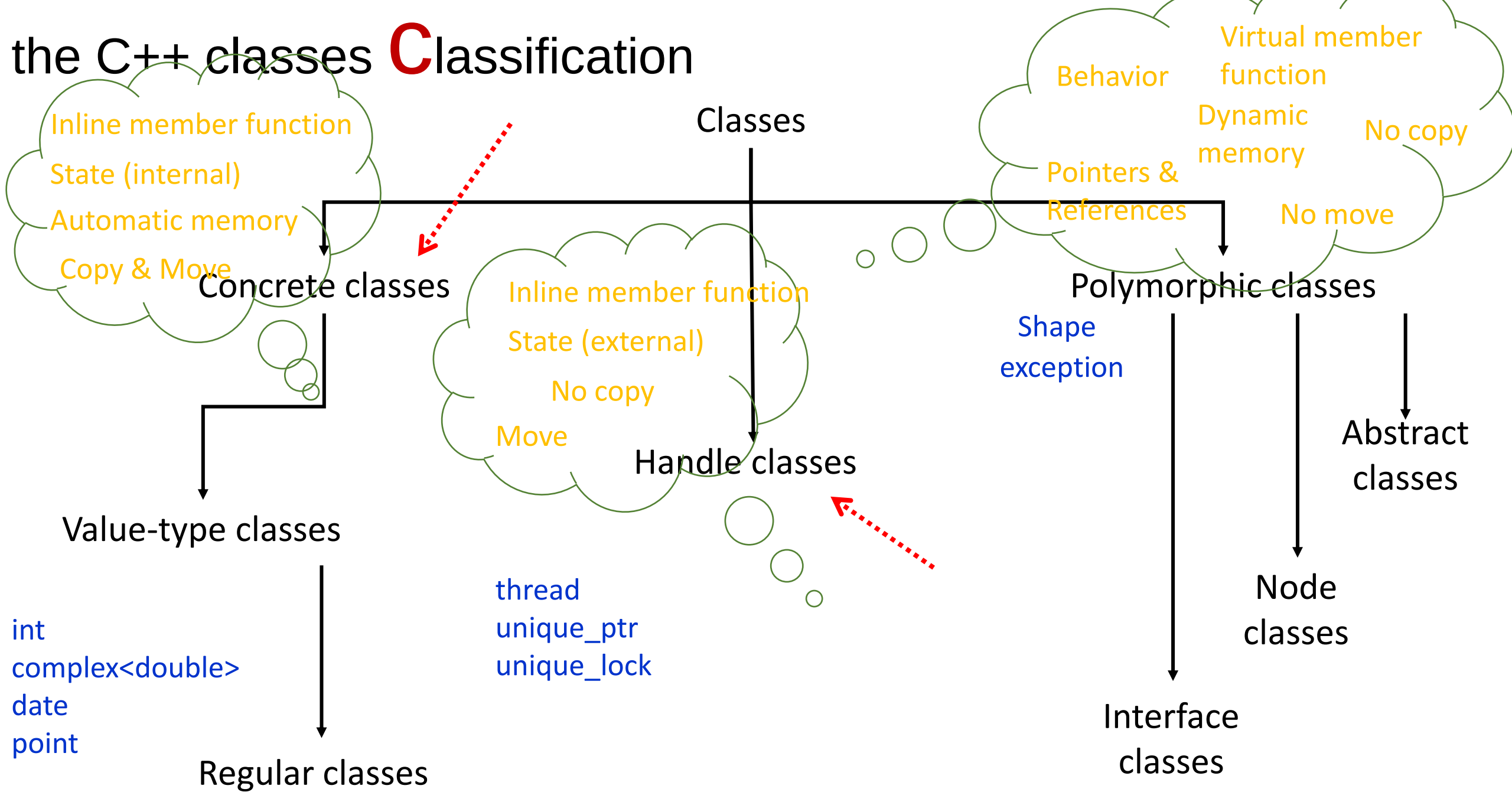
A class is the mechanism for representing concepts.



Cclasses: benefits

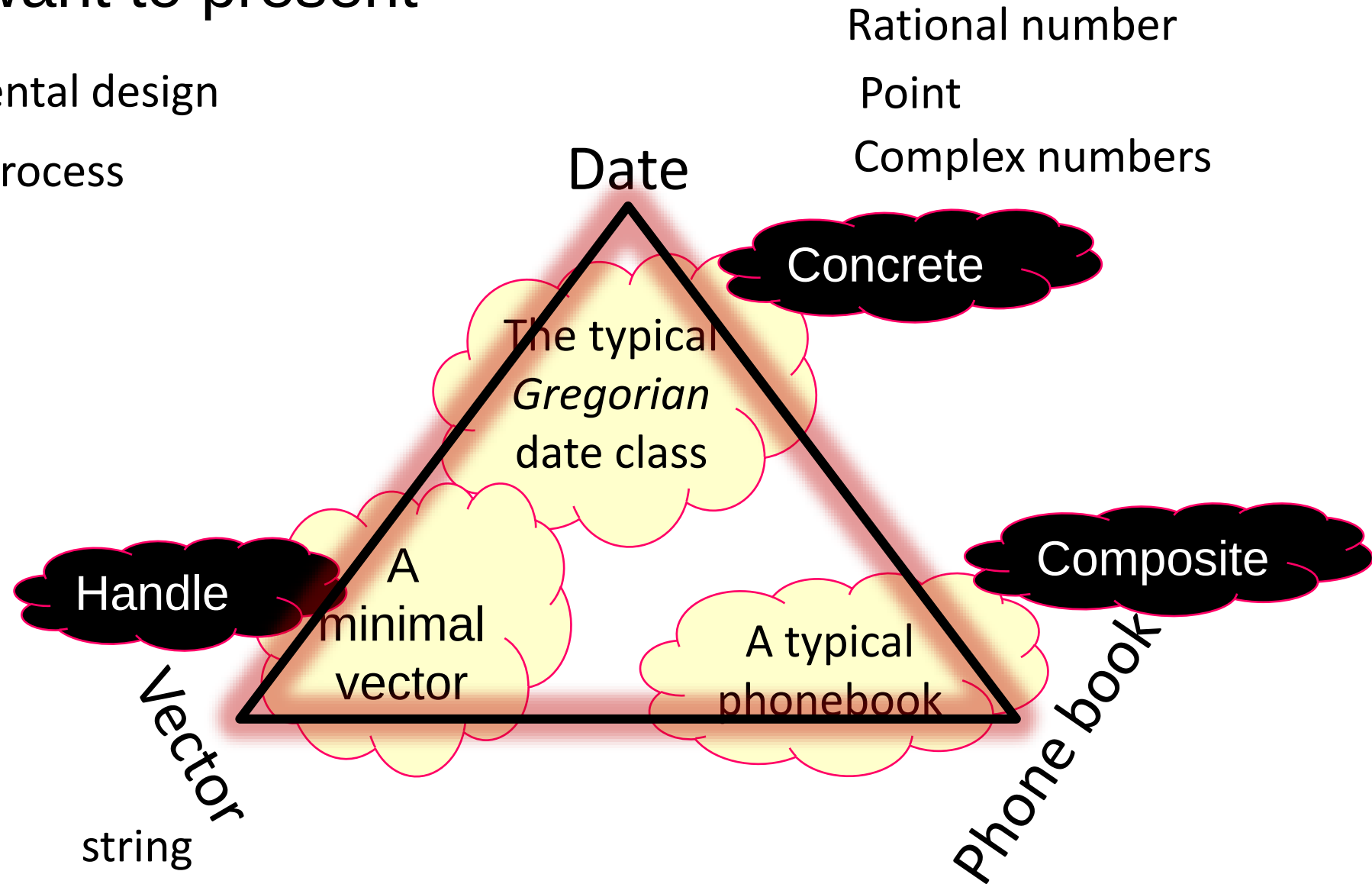
- A program that provides types that closely match the concepts of the application tends to be:
 - easier to understand
 - easier to reason about
 - easier to modify them
- A well-chosen set of user-defined types makes:
 - a program more concise
 - many sorts of code analysis feasible
- Fundamental idea: separating incidental details of the implementation from essential properties.
- Better expressiveness
- Software reuse

the C++ classes Classification



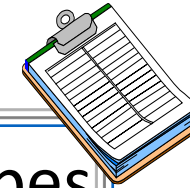
the **C**lasses we want to present

- The iterative and incremental design
- The gradual refinement process



Concrete classes: introduction

- a concrete type resembles a built-in type.



Language-technical
rule

Provide as good support for user-defined types
as for built-in types.

```
void f()
{
    int i; // int variable declaration/definition
    int j = i; // initialize an int variable
    j = i; // assign new value to already defined int

    int* pi = &i; // take the address of int variable
    int* p = new int(0); // make -unnamed- int in heap
    // ...
    delete p; // release the memory allocated by var.
}
```

The Date Class

1. Separated states and behaviors

- Data members

```
struct Date { // representation
    int day, month, year;
}; // declaration

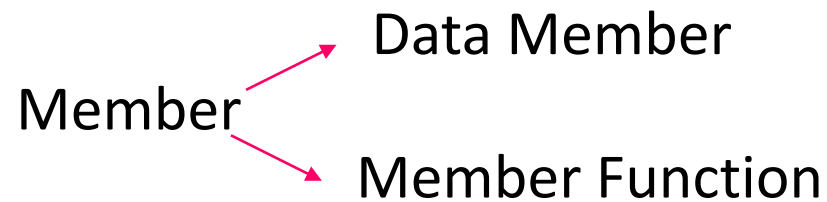
// operations
void init_date(Date& d, int, int, int); // initialize d
void add_year(Date& d, int); // add n years to d
void add_month(Date& d, int n); // add n months to d
void add_day(Date& d, int n); // add n days to d
// ...
```

```
void init_date(Date& dd, int d, int m, int y)
{
    dd.day = d; dd.month = m; dd.year = y;
}
// other functions
```

Member functions
Object inversion

- struct is a kind of class.
- There is no explicit connection between the data type and these functions.

2. Member Functions



```
class X {
    data members
    member functions
};
```

```
struct Date {
    int day, month, year; // data members
    // member functions
    void init(int, int, int);
    void add_year(int);
    void add_month(int n);
    void add_day(int n);
    // ...
};
```

- Old terminology: Object inversion

The **V**ector Class

- The very 1st try: fixed-size and struct-based vector

R eferences and further readings

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

